

AppForge AI

A Dynamic Application Generator

1. Project Overview

AppForge AI is an advanced systems-level application generator that takes structured configurations (JSON) and converts them into fully functional, deployed web applications. It serves as a dynamic runtime engine rather than a static codebase, meaning that multiple "apps" can be hosted, managed, and rendered through a single cohesive system.

Core Philosophy

The core principle behind AppForge AI is **Config-Driven Development**. Instead of hardcoding components, endpoints, and schemas, everything is derived from a single source of truth: the JSON configuration. This configuration defines:

- **Entities:** The data models (e.g., Tasks, Projects, Users).
- **Pages:** The UI views (e.g., Tables, Forms, Dashboards).
- **Auth:** Authentication rules.
- **Localization:** Multi-language support.
- **Notifications:** Event-driven alerts.

2. Architecture & Tech Stack

Frontend

- **Framework:** React.js & Next.js (App Router).
- **Styling:** Vanilla CSS with modern aesthetics (glassmorphism, gradients, CSS variables).
- **Animations:** Framer Motion for micro-animations and page transitions.
- **State Management:** Zustand for global state (Auth, Locale, Notifications) with persistence.

Backend

- **Environment:** Node.js (via Next.js API Routes).
- **Language:** TypeScript for strong typing and error prevention.
- **API Architecture:** RESTful dynamic endpoints that handle any generated entity.

Database

- **Engine:** SQLite (via `better-sqlite3`).
- **Storage:** A meta-schema approach.
 - `apps`: Stores the application configuration.
 - `app_data`: Stores records for any entity inside any app using JSON stringification (NoSQL behavior inside a SQL database).
 - `dynamic_schemas`: Stores the schemas for quick validation.

Authentication

- **System:** JWT (JSON Web Tokens) and bcrypt.
- **Access Control:** User-scoped app isolation. A user can only see, edit, and interact with applications and data they have created.

3. How the System Works (The Runtime)

1. The Configuration Engine (`src/lib/config-engine.ts`)

When a user submits a JSON configuration, the **Config Engine** takes over.

- It parses the JSON.
- It normalizes incomplete data (e.g., if a field type is missing, it defaults to `text`).
- It generates default pages if none are provided (e.g., a table page for every entity).
- It validates the structure to prevent system crashes.

2. The Dynamic API Layer

Instead of creating a new API file for every entity, AppForge uses wildcard routes:

- `POST /api/apps/[appId]/data/[entity]`
- `GET /api/apps/[appId]/data/[entity]`
These routes dynamically look up the entity schema from the Config Engine, validate the incoming payload against the schema rules (e.g., required fields, min/max values), and interact with the database.

3. The Dynamic UI Renderer (`src/app/app/[appId]/page.tsx`)

This is the heart of the system. When a user opens a generated app:

1. The page fetches the JSON config for that specific `appId`.
2. It dynamically renders a sidebar navigation based on the `pages` array.
3. If the user clicks a "Table" page, it renders a table by iterating over the `entity.fields` array to generate columns, and fetches the data from the dynamic API.
4. If the user clicks "Add Record", a dynamic form is generated on the fly. A `checkbox` type becomes a literal checkbox, a `textarea` becomes a text box, etc.

4. Key Features Implemented

1. Multi-Language / Localization

- **Implementation:** The JSON config can contain a `locales` object mapping strings to different languages.
- **Usage:** A lightweight `i18n.ts` utility translates keys on the fly. Users can switch their global locale from the dashboard header, instantly translating the dynamic apps.

2. CSV Import System

- **Implementation:** `PapaParse` is used to read CSV files directly from the UI.
- **Usage:** Users can open an entity table and click "Import CSV". The system maps the CSV columns to the entity fields, coerces types (e.g., parsing strings into numbers or booleans), and bulk-inserts the records into the database.

3. Event-Driven Notifications

- **Implementation:** A global `notifications` table and a Zustand store.
- **Usage:** When critical events occur (e.g., App Creation, CSV Import completion, or Record creation), the API drops a notification into the database. The UI checks this and displays an unread badge on the bell icon in the dashboard.

5. Edge Case Handling & Reliability

AppForge AI is designed to never break, even when given bad data:

- **Incomplete Configurations:** If an entity lacks pages, the system auto-generates them. If a field lacks a label, the system capitalizes the field name.
- **Schema Mismatches:** If old data exists in the database but the user updates the JSON config to add a new field, the system gracefully fills the missing field with the defined `default` value during read/write operations.
- **Invalid CSV Data:** If a row in a CSV fails validation, the transaction isolates that row, skips it, and continues importing the rest, returning a detailed error report of skipped rows.

6. Running the Application

1. **Install Dependencies:** `npm install`
2. **Start Development Server:** `npm run dev`
3. **Usage Flow:**
 - Navigate to `http://localhost:3000`.
 - Click "Get Started" and create an account.
 - On the Dashboard, click "+ New App".
 - Click "Use Sample Config" to see a complex configuration (Task Manager Pro) populated automatically.
 - Click "Create App" and open it to interact with your dynamically generated software.